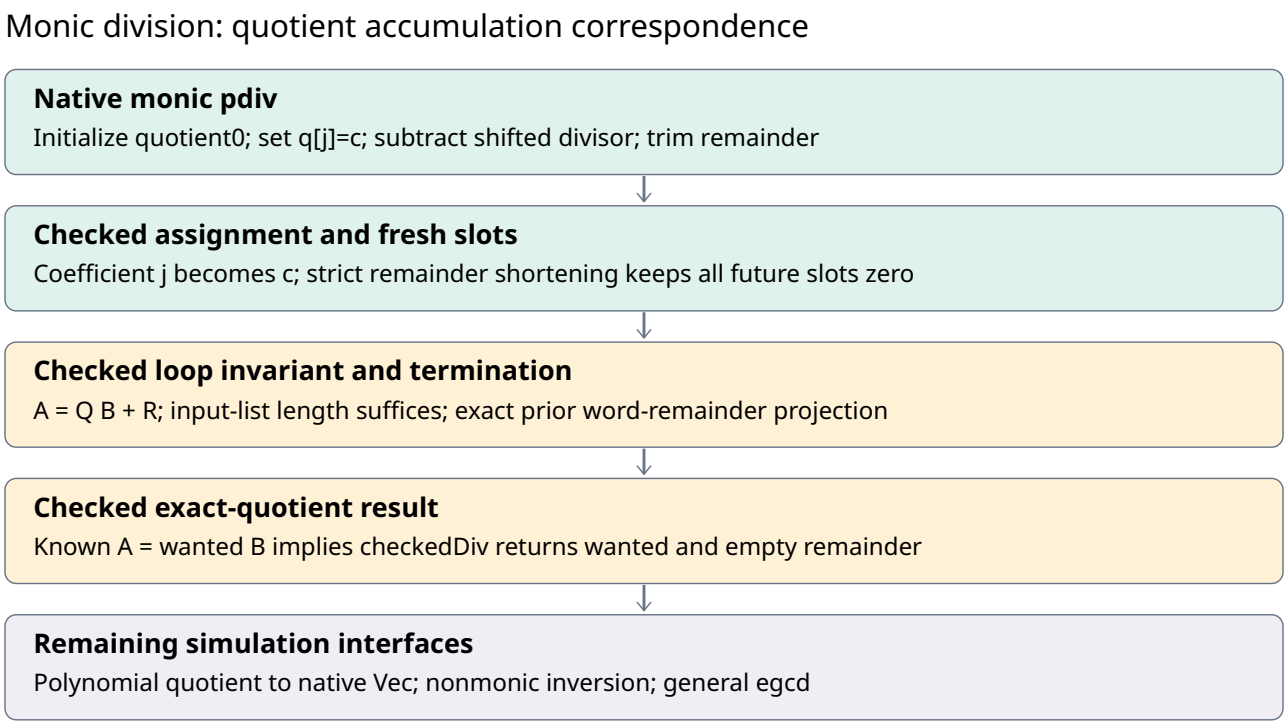
Monic native-division quotient correspondence



pencil-division-quotient-20261101-v1 | 17 main + 9 controls | no scientific measurements

Seventeen main declarations and nine controls passed, with twenty-six named axiom readbacks. The construction now carries the quotient through the actual checked word-remainder loop, proves that every quotient coefficient assignment is to a fresh slot, preserves A=Q*B+R, and returns the unique exact quotient when the input is divisible. This advances the CZero Cantor route from an exact-factorization certificate to a terminating quotient-accumulation model.

Source basis. Native three_leads.rs849-880 asserts a nonempty divisor, trims the input remainder, allocates a zero quotient vector, selects the inverse leading coefficient, and repeatedly assigns q[j]=c before subtracting the shifted multiple from r and trimming. In the monic branch the inverse is1, j=r.length-b.length and c is the last word of r. Native pexact885-889 then requires the final remainder to be empty. Native pegcd900-926 and reduce990-1003 use those helpers. Exact source bytes and line-numbered excerpts are bound in source.json and native-source.rs.txt.

Literal assignment semantics. QuotientWrite.writeCoeff(q,j,c) is q+C(c-q.coeff(j))*X^j. The checked coefficient laws prove that slot j becomes c and all other slots remain unchanged. This models assignment, not unconditional addition. Fresh(b,r,q) says every slot i with i+length(b)<=length(r) is zero. It holds for the initial zero polynomial. The imported strict shortening theorem for the selected word-remainder step proves the next Fresh invariant. Consequently each actual write satisfies q.coeff(j)=0, so it equals q+C(c)*X^j. The quotient update and remainder subtraction then cancel in Q*B+R. A separate checked natural-number bound shows the selected index fits the native initial quotient allocation whenever r.length<=a.length and b.length<=r.length.

Loop correspondence. DivisionLoop.qloop repeats the existing MonicRemainder.selected function, including its UInt64 coefficient traversal and trim, while carrying the polynomial quotient. It preserves canonical remainder words, the fresh-slot invariant and Q*B+R, and success ends only with an empty or shorter remainder. A structural proof derives successful execution whenever fuel is at least the input remainder length, using the existing strict word-list length decrease. The remainder projection is exactly the earlier MonicRemainder.loop, rather than an unrelated remainder specification. Fuel is a proof-model bound; the native loop has no fuel argument.

Exactness interface. checkedDiv admits canonical monic divisors through last-word=1, starts with quotient0 and the trimmed input, and projects exactly to checkedPrem. checked_correct gives wordsPoly(a)=Q*wordsPoly(b)+wordsPoly(out). exact_iff_divisible proves that some quotient with empty remainder is returned exactly when the decoded divisor divides the decoded input, given sufficient fuel. quotient_unique identifies that returned polynomial with any supplied exact factor. Finally returns_given_exact proves the algorithmic result checkedDiv(...)=some(wanted,[]) from a known exact factorization, rather than assuming the loop returned wanted. This is the interface for substituting the previously checked CZero quotient X-x.

Assumptions and machine boundary. The decoded coefficient field is ZMod(p.toNat), with prime p, positive p and p<2^31; both input word lists are canonical. The admitted divisor ends in literal word1. The existing remainder traversal proofs carry the canonical scalar arithmetic and no-overflow bound. The quotient is represented here as a polynomial, and its coefficient overwrites are proved, but no full simulation from that polynomial to the native quotient Vec allocation, initialized storage, indexing writes and final trim is claimed. The natural-number index bound is one component of that simulation. Native nonmonic leading-inverse handling, general pegcd Bezout invariants/termination, instrumentation and compiled equivalence remain separate interfaces. checkedDiv's rejection of nonmonic input narrows this proof's admission; native pdiv itself supports nonmonic divisors.

Controls. Over ZMod17 the actual two-step model divides [16,0,1] by [16,1], returning X+1 and empty remainder; [1,0,1] returns X+1 with remainder[2]. Zero input is handled after trim. Zero fuel and one insufficient step return none for the active two-step case. Empty and nonmonic divisors are rejected by this monic model. A used quotient slot demonstrates the load-bearing distinction: replacing coefficient2 by3 produces3, whereas adding3 would produce5. The fresh-slot proof is what makes addition legitimate along this particular loop. These are kernel-checked controls, not scientific measurements.

Verification and failures. Final gates are write2, loop1, exact2, interface1, controls2 and readback1; all returned0 and preserved input bindings. The explicit inventory contains seventeen main declarations and nine controls, each with its own axiom readback. Only propext, Classical.choice and Quot.sound occur. Failed write1 needed explicit coefficient simplification before map expansion; failed exact1 needed the listPoly empty definition; failed controls1 needed a decidable ZMod inequality after numeric simplification. Every failed source/log/receipt is retained. The fail-closed helper records compiler exits and unchanged bindings in fresh attempt directories. Cached transitive dependencies and the compiler remain trusted; no independent binary kernel rechecker was run.

Provenance and next composition. The PLAN was written after native source and dependency inspection and before new proof code. The source basis is native pdiv plus the existing selected_correct, selected_shorter, checked_empty_iff and decoded polynomial interfaces; no external empirical result is invoked. This is a producer packet awaiting independent source-fidelity review after publication. Its next direct use is to combine returns_given_exact with the CZero factorization under matched word encodings, then finish quotient-array simulation and the general egcd bridge. No repository edits, scientific runs, timing measurements or cloud jobs were performed.